

Architettura dei Calcolatori

Terzo Appello – 22 luglio 2022 – 1h45

1. [7 pt] Dato il seguente snapshot della memoria istruzioni, contenente un frammento di codice binario RISC-V

```
0x4000 0x00A904B3
0x4004 0x00B48023
0x4008 0x00040A67
0x400C 0x00190913
0x4010 0xFF1FF06F
0x4014 0x00008067
```

Si decodifichino le istruzioni nell'equivalente codice assembly. Dove sono implicate istruzioni di alterazione del control flow si trasformino gli offset di salto in opportune label.

beq	x8, x0, 0x00A	[2pt]	L1: beq x8, x0, L2
add	x9, x18, x10	[.75pt]	add x9, x18, x10
sb	x11, 0(x9)	[.75pt]	sb x11, 0(x9)
addi	x18, x18, 1	[.75pt]	addi x18, x18, 1
jal	x0, 0xFFFF8	[2pt]	jal x0, L1
jalr	x0, 0(x1)	[.75pt]	L2: jalr x0, 0(x1)

2. [6 pt] Implementare in assembly RISC-V la funzione C **strcat**

```
char *strcat(char *dest, const char *src);
```

La funzione `strcat()` concatena la stringa `src` aggiungendola al termine della stringa `dest`. La concatenazione avviene copiando la stringa `src` a partire dal terminatore della stringa `dest`. La copia di `src` e' comprensiva del suo terminatore. L'area di memoria puntata da `dest` deve essere sufficientemente ampia da accogliere entrambe le stringhe ed il terminatore `'\0'`, quindi `dest` deve essere dimensionata almeno per contenere `strlen(dest) + strlen(src) + 1` caratteri.

Gestire correttamente lo stack come da calling convention, così come il passaggio dei parametri di ingresso e di uscita sui registri appropriati. Scrivere la funzione `main` che invoca **strcat**, mostrando anche l'allocazione degli array sul segmento dati statico del file ELF.

- Main e dati 1pt
- 1pt per la gestione dello stack
- 4pt per la corretta implementazione della copia principale

SOLUZIONE // a0 = dest; a1 = src;

strcat:

```
addi sp,sp,-32      // adjust stack for 4 dw
sd    s0,0(sp)      // push s0
sd    s1,8(sp)      // push s1
sd    s2,16(sp)     // push s2
sd    s3,24(sp)     // push s3
```

```
li    s2,0          // i = 0
li    s3,0          // j = 0
```

posizioniamoci sul terminatore di dest

```
L1:   add    s1,s2,a0      // s1 = addr of dest[i]
      lbu    s1,0(s1)     // s1 = dest[i]
      beq    s1,zero,L2   // if dest[i] == 0 then proceed to copy
      addi   s2,s2,1      // i = i + 1
      j      L1           // next iteration of loop
```

s2 (i) indicizza dest, s3 (j) indicizza src

```
L2:   add    s0,s3,a1      // s0 = addr of src[j]
      lb     s0,0(s0)     // s0 = src[j]
      add    s1,s2,a0      // s1 = addr of dest[i]
      sb     s0,0(s1)     // dest[i] = src[j]
      beq    s0,zero,L3   // if src[i] == 0 then exit
      addi   s2,s2,1      // i = i + 1
      addi   s3,s3,1      // j = j + 1
      j      L2           // next iteration of loop
```

```
L3:   ld     s0,0(sp)     // restore saved s2
      ld     s1,8(sp)     // restore saved s2
      ld     s2,16(sp)    // restore saved s2
      ld     s3,24(sp)    // restore saved s3
      addi   sp,sp,32     // pop 4 dw from stack
      ret
```

3. Si consideri una cache che usa 8 bit per il campo offset e 4 bit per il campo index. Ipotizzando di aver già letto l'indirizzo 0x0100, quali affermazioni sono vere (spiegare il ragionamento per arrivare alla risposta)?
- Se eseguire letture agli indirizzi 0x0114, 0x0308, 0x1105 produce due conflict miss e una cold cache miss allora la cache è direct mapped
 - Se la stessa sequenza di letture del punto a) produce tutte cold cache miss allora la cache è set-associative, con almeno 4 vie
 - Se la stessa sequenza di letture del punto a) produce due cold cache miss e una conflict miss allora la cache è 2-way set-associative
 - Se la cache è direct mapped la sua dimensione è 4KB

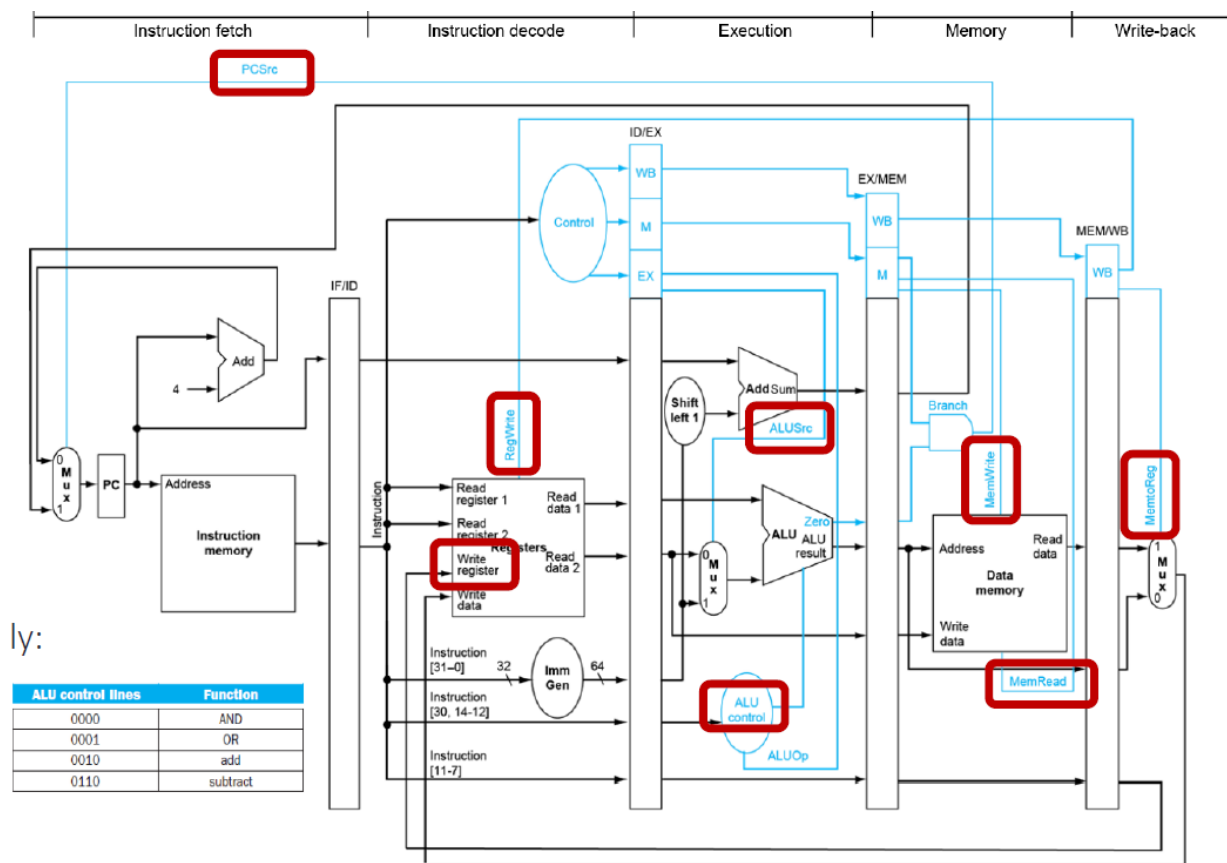
	TAG	INDEX	OFFSET	direct	2-way	4-way
0x0100	0...0000	0001	0000 0000	CCM	CCM	CCM
0x0114	0...0000	0001	0001 0100	hit	hit	hit
0x0308	0...0000	0011	0000 1000	CCM	CCM	CCM
0x1105	0...0001	0001	0000 0101	CM	CCM	CCM

- FALSA.** Per una DM la sequenza di letture produce una hit, una cold cache miss e una conflict miss.
- FALSA.** Per una 4-way set-associative la sequenza produce una hit e due CCM.
- FALSA.** Per una 2-way set-associative la sequenza produce una hit e due CCM.
- VERA.** Ho 2^4 linee da 2^{16} bytes ciascuna, da cui $16 \times 256 = 4096$ Bytes.

4. [5 pt] Si consideri il diagramma della pipeline RISC-V single-cycle indicato in figura. Sapendo che i segnali di controllo cerchiati in rosso valgono

PCSrc = 1
 RegWrite = 1
 Write register = 10
 ALUSrc = 1
 ALU control = 0110
 MemWrite = 1
 MemRead = 0
 MemtoReg = 1

dire che istruzioni si trovano nei vari stadi della pipeline, ove possibile. Per ogni istruzione si fornisca il massimo livello di dettaglio possibile (es., in certi casi si potrà solo dire a che classe appartiene una istruzione, in altri si potrà identificare esattamente l'istruzione. In alcuni casi non si potrà dire niente sui registri utilizzati dall'istruzione, in altri sì; etc.).



Se PCSrc = 1 di sicuro in fase di memory c'è una branch, e la sua condizione è TRUE.

Da RegWrite = 1 e Write register = 10 so che l'istruzione in WB scrive su x10. Dato che MemtoReg = 1 posso dire che si tratta di una LOAD x10, ...

Dato che MemWrite = 1 posso stabilire che nella fase di memory ci sia una STORE. NOTA: Questo è però in contrasto con quanto stabilito grazie al PCSrc.

Per gli altri stadi non sono in grado di dire niente.

5. [5 pt] Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e dynamic branch predictor a due bit, inizialmente settato a BRANCH NOT TAKEN.

```

    li    t2,3 // UB
    li    t0,1 // i = 1
loop_i
    li    t1,0 // j = 0
loop_j
    addi  t1,t1,1
    blt   t1,t2,loop_j
    addi  t0,t0,1
    blt   t0,t2,loop_i

    jr    ra

```

- a) Si calcoli la misprediction rate [2,5 pt].
b) Sapendo che:
- per ogni predizione corretta un'istruzione di tipo branch impiega 1 ciclo ad eseguire, mentre in caso di misprediction ne impiega 8
 - le istruzioni aritmetico-logiche impiegano 3 cicli a eseguire
 - il salto incondizionato impiega 1 ciclo ad eseguire
- Si calcoli il CPI medio per l'esecuzione del programma [2,5 pt]

	istanza di istruzione	registri e condizioni	esito salto	2bit pred		miss#		
1)	li t0, 1	t0 = 1						
2)	li t2, 3	t2 = 3						
3)	li t1, 0	t1 = 0						
4)	addi t1, t1, 1	t1 = 1						
5)	blt t1, t2, loop_j	t1 < t2 → 1 < 3	T	NTs	M	[1]	Ts	H
6)	addi t1, t1, 1	t1 = 2						
7)	blt t1, t2, loop_j	t1 < t2 → 2 < 3	T	NTm	M	[2]	Ts	H
8)	addi t1, t1, 1	t1 = 3						
9)	blt t1, t2, loop_j	t1 < t2 → 3 < 3	N	Tm	M	[3]	Ts	M
10)	addi t0, t0, 1	t0 = 2						
11)	blt t0, t2, loop_i	t0 < t2 → 1 < 3	T	NTm	M	[4]	Tm	H
12)	li t1, 0	t1 = 0						
13)	addi t1, t1, 1	t1 = 1						
14)	blt t1, t2, loop_j	t1 < t2 → 1 < 3	T	Tm	H	[5]	Ts	H
15)	addi t1, t1, 1	t1 = 2						
16)	blt t1, t2, loop_j	t1 < t2 → 2 < 3	T	Ts	H	[6]	Ts	H
17)	addi t1, t1, 1	t1 = 3						
18)	blt t1, t2, loop_j	t1 < t2 → 3 < 3	N	Ts	M	[7]	Ts	M
19)	addi t0, t0, 1	t0 = 3						
20)	blt t0, t2, loop_i	t0 < t2 → 3 < 3	N	Tm	M	[8]	Tm	M
21)	jr ra							

Su 21 istruzioni totali 8 sono branch, 12 aritmetiche (CPI=3) e 1 salto incondizionato (CPI=1). Per il predittore a due bit la misprediction rate è 6/8. Quindi il CPI medio per il programma è

$$CPI = \frac{12 I_{ALU} * 3 \frac{cicli}{I_{ALU}} + 1 I_{JMP} * 1 \frac{cicli}{I_{JMP}} + 8 I_{BR} \left(\frac{6 mis * 8 \frac{cicli}{mis} + 2 pr_{ok} * 1 \frac{cicli}{pr_{ok}}}{8} \right)}{21 I} = \frac{36 + 1 + 8 \left(\frac{48 + 2}{8} \right)}{21} = \frac{87}{21} \cong 4$$

6. **[6 pt]** Si progetti una rete logica sequenziale con un segnale di ingresso X a 1 bit e un segnale di uscita Z a 1 bit. La rete manda alta l'uscita quando vengono ricevuti quattro zeri in ingresso (indipendentemente dal numero di '1' ricevuti), come mostrato nell'esempio sottostante.

ciclo	0	1	2	3	4	5	6	7	8	9	10	11
X	0	1	1	0	0	1	0	1	0	0	0	0
Z	0	0	0	0	0	0	1	0	0	0	0	1

L'ultimo '0' di una sequenza NON è contato come il primo '0' di una nuova sequenza).

- Determinare il numero minimo di stati necessari per risolvere il problema, scegliendo Mealy o Moore come si ritiene più opportuno. Disegnare il diagramma degli stati **[2 pt]**.
- Suggerire una codifica per gli stati e derivare la tabella di verità. Da lì popolare le mappe di Karnaugh e ricavare le forme minime per le reti di stato futuro e per le uscite **[3 pt]**.
- Disegnare il circuito finale **[1 pt]**.

Appendice: Tabelle di codifica delle istruzioni RISC-V

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lr.d	0110011	011	0001000
	sc.d	0110011	011	0001100
Format	Instruction	Opcode	Funct3	Funct6/7
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	ld	0000011	011	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	lwu	0000011	110	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	slli	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
Format	Instruction	Opcode	Funct3	Funct6/7
S-type	sb	0100011	000	n.a.
	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sd	0100011	111	n.a.
SB-type	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
U-type	lui	0110111	n.a.	n.a.
UJ-type	jal	1101111	n.a.	n.a.

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value